

Department of Electronic & Telecommunication Engineering
University of Moratuwa, Sri Lanka

EN4554 Deep Learning for Vision

Assignment 2: Convolutional Neural Networks

Nadha M.I.
220409J

GitHub: `https://github.com/NadhaIrshad/EN4554-Deep-Learning-for-Vision`

Date: 16.10.2026

Setup. All experiments use PyTorch on a Colab T4 GPU. The dataset is CIFAR-10, loaded with `torchvision.datasets.CIFAR10`. The 50k training images are split with a fixed permutation into 45k training and 5k validation images. Each channel is normalised with the training-set statistics (mean $(0.491, 0.482, 0.446)$, std $(0.247, 0.244, 0.262)$). All random seeds are fixed. The test set is used only once, in Task 4. Data augmentation (flip, crop, cutout, mixup) is implemented by hand and applied to whole mini-batches on the GPU. Training accuracy is always re-measured on clean (non-augmented) training images in eval mode, so train/validation gaps are compared fairly.

AI-tool acknowledgement. Claude (Anthropic, via Claude Code) was used to help write and debug the code. I have read and understood all of it.

1 Task 1: Backpropagation

(a) Layers with explicit forward/backward

Each layer caches its input in `forward` and returns $\partial L/\partial(\text{input})$ in `backward`. For $Y = XW + b$ with upstream gradient G: $\partial L/\partial W = X^\top G$, $\partial L/\partial b = \sum_n G_n$, $\partial L/\partial X = GW^\top$. For softmax cross-entropy averaged over N samples: $\partial L/\partial z = (\text{softmax}(z) - \text{onehot}(y))/N$.

```
class Linear:
    def forward(self, x):
        self.x = x
        return x @ self.W + self.b
    def backward(self, g):
        self.dW = self.x.T @ g
        self.db = g.sum(0)
        return g @ self.W.T

class ReLU:
    def forward(self, x):
        self.mask = (x > 0).to(x.dtype); return x * self.mask
    def backward(self, g):
        return g * self.mask

class SoftmaxCrossEntropy:
    def forward(self, z, y):
        z = z - z.max(dim=1, keepdim=True).values                    # numerical
    stability
        logp = z - torch.log(torch.exp(z).sum(dim=1, keepdim=True))
        self.p, self.y = torch.exp(logp), y
        return -logp[torch.arange(len(y)), y].mean()
    def backward(self):
        g = self.p.clone(); g[torch.arange(len(self.y)), self.y] -= 1.0
        return g / len(self.y)
```

The `MLP` class stacks `Linear` and activation layers, calls `backward` in reverse order, and updates $W \leftarrow W - \eta\, \partial L/\partial W$.

(b) Gradient check

The check uses a float64 MLP (32–24–16–10, ReLU) on 8 random inputs, so every parameter can be checked with centred differences $\frac{L(\theta+\epsilon) - L(\theta-\epsilon)}{2\epsilon}$, $\epsilon = 10^{-6}$. The relative error is

$\|g_1 - g_2\|_2 / (\|g_1\|_2 + \|g_2\|_2)$.

Tensor	Rel. err. vs autograd	`allclose`	Rel. err. vs finite diff.	`allclose`
W_1 (32×24)	0	True	1.37×10^{-9}	True
b_1	0	True	8.82×10^{-10}	True
W_2 (24×16)	0	True	7.62×10^{-10}	True
b_2	0	True	9.88×10^{-10}	True
W_3 (16×10)	0	True	4.79×10^{-10}	True
b_3	0	True	8.30×10^{-10}	True
input x	0	True	3.15×10^{-9}	True

Table 1: Gradient check. The full CIFAR MLP (3072–512–256–10, float32, real images) also matches autograd with relative error 0 and `allclose`=True for all layers.

Interpretation. The analytic gradients agree with autograd to machine precision, as expected, because both perform the same matrix operations. The finite-difference errors ($\sim 10^{-9}$) are at the level expected from truncation ($O(\epsilon^2)$) plus round-off ($\sim \varepsilon_{\text{mach}}/\epsilon$) in float64. A real bug, such as a missing transpose or a missing $1/N$, would give errors of order 10^{-1}.

(c) Training the MLP with SGD

The 3072–512–256–10 ReLU MLP (He initialisation) is trained on flattened images with plain SGD (lr = 0.02, batch 128, 20 epochs), without autograd.

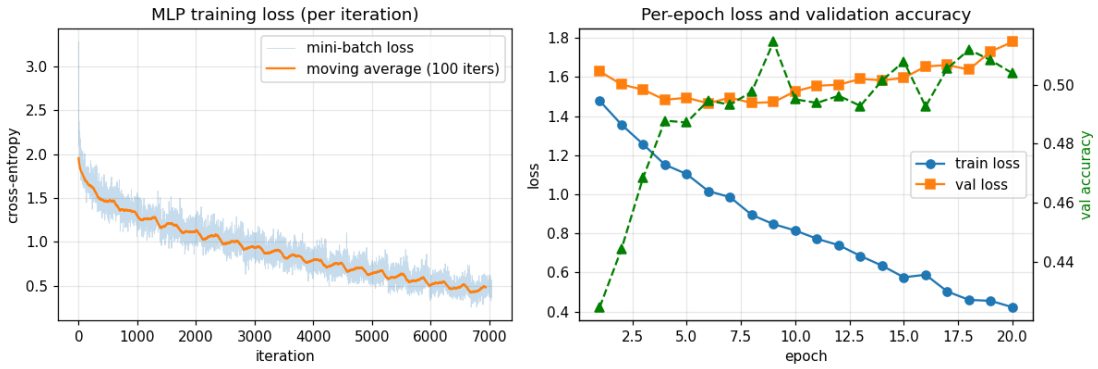


Figure 1: Left: mini-batch training loss. Right: per-epoch train/validation loss and validation accuracy.

Validation accuracy: 50.4% after 20 epochs (best 51.4% at epoch 9). Interpretation. The network learns quickly (42.5% after one epoch, far above the 10% chance level), but after about epoch 6–9 it overfits. Training loss keeps falling (1.48 → 0.42, 88% train accuracy), while validation loss rises from about 1.47 to 1.78. A fully connected net on raw pixels has no locality or translation invariance, and with about 1.7M parameters it memorises pixel patterns that do not generalise. This limits it to about 50%, whereas the CNNs below reach 70–89%.

(d) Gradient norms in a 10-layer MLP at initialisation

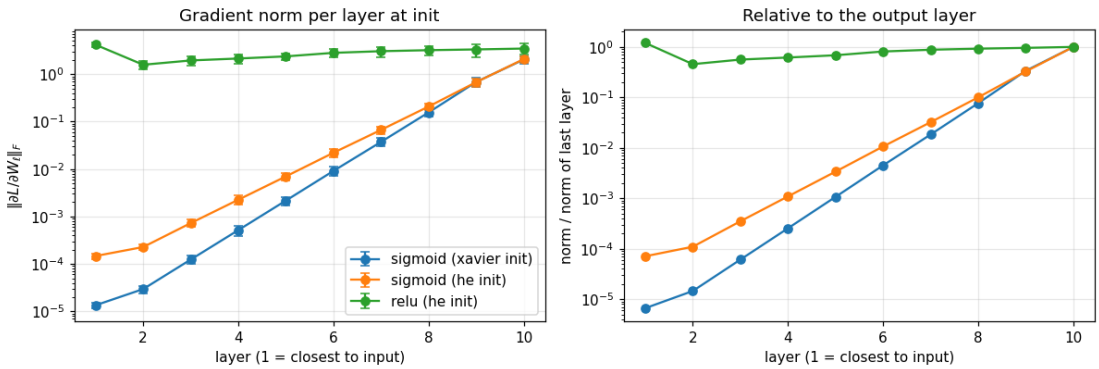


Figure 2: $\|\partial L/\partial W_\ell\|_F$ per layer at initialisation (mean $\pm$ std over 5 seeds, batch of 256 images).

Interpretation. With sigmoid, the gradient norm decays geometrically towards the input. Layer 1 receives only 6.6×10^{-6} of the last layer's gradient norm (1.35×10^{-5} vs 2.04), so the early layers barely learn: the *vanishing gradient* problem. Backpropagation multiplies by $W_\ell^\top \mathrm{diag}(\sigma'(z_\ell))$ at each layer, and $\sigma'(z) \leq 0.25$, so nine hidden layers give roughly $4^{-9} \approx 4 \times 10^{-6}$, which matches the measurement. With ReLU and He initialisation the profile is almost flat (ratio ≈ 1.2). ReLU has derivative 1 on its active half, and $\mathrm{Var}(W) = 2/n_{\text{in}}$ compensates exactly for the half that is zeroed, so the variance of the backward signal is preserved. Sigmoid with He initialisation is only about $10\times$ better, which confirms that the activation, not the initialisation, is the cause.

2 Task 2: Optimizers

(a) SGD with momentum and Adam as `torch.optim.Optimizer` subclasses

```
class MySGD(torch.optim.Optimizer):          # b_t = mu*b_{t-1} + g_t ;  theta -= lr*
    b_t
    @torch.no_grad()
    def step(self, closure=None):
        for group in self.param_groups:
            for p in group['params']:
                if p.grad is None: continue
                d = p.grad
                if group['weight_decay'] != 0: d = d.add(p, alpha=group['weight_decay'
    ])
                if group['momentum'] != 0:
                    state = self.state[p]
                    if 'buf' not in state: state['buf'] = d.clone()
                    else: state['buf'].mul_(group['momentum']).add_(d)
                    d = state['buf']
                p.add_(d, alpha=-group['lr'])

class MyAdam(torch.optim.Optimizer):
    @torch.no_grad()
```

```
    def step(self, closure=None):
        for group in self.param_groups:
            b1, b2 = group['betas']
            for p in group['params']:
                if p.grad is None: continue
                g = p.grad if group['weight_decay'] == 0 else p.grad.add(p, alpha=
    group['weight_decay'])
                st = self.state[p]
                if not st: st['t'], st['m'], st['v'] = 0, torch.zeros_like(p), torch.
    zeros_like(p)
                st['t'] += 1; t, m, v = st['t'], st['m'], st['v']
                m.mul_(b1).add_(g, alpha=1 - b1)
                v.mul_(b2).addcmul_(g, g, value=1 - b2)
                m_hat, v_hat = m / (1 - b1 ** t), v / (1 - b2 ** t)       # bias
    correction
                p.addcdiv_(m_hat, v_hat.sqrt().add_(group['eps']), value=-group['lr'])
```

Setting (50 steps, small CNN, float64)	$\max \lvert\theta_{\text{mine}} - \theta_{\text{torch}}\rvert$	`allclose`
SGD, lr $= 0.05$	1.39×10^{-16}	True
SGD + momentum 0.9, wd 5×10^{-4}	1.11×10^{-16}	True
Adam, lr $= 10^{-3}$	2.78×10^{-17}	True
Adam, lr $= 3 \times 10^{-3}$, wd 10^{-4}	8.33×10^{-17}	True

Table 2: My optimisers vs. `torch.optim.SGD` / `torch.optim.Adam`.

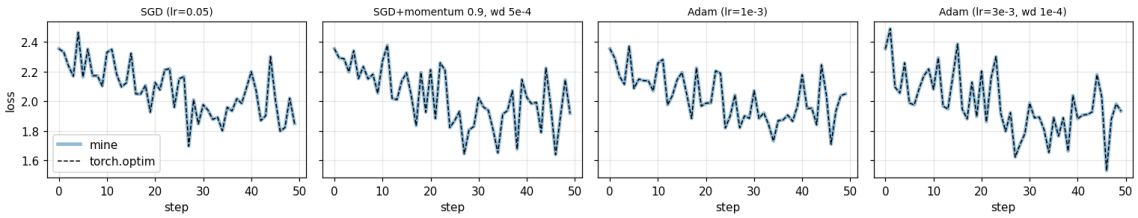


Figure 3: Loss curves of my optimisers (thick) and `torch.optim` (dashed) coincide.

Interpretation. The parameters match to float64 round-off ($\sim 10^{-16}$). $\hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon)$ is algebraically identical to PyTorch's $\frac{m_t/(1-\beta_1^t)}{\sqrt{v_t}/\sqrt{1-\beta_2^t}+\epsilon}$, so the only differences come from floating-point evaluation order. Bias correction matters early in training. Without it, $m_1 = 0.1g_1$ and $v_1 = 0.001g_1^2$, so the first steps would be about $3\times$ too small.

(b) Trajectories on $f(x, y) = \frac{1}{2}(x^2 + 50y^2)$

All runs start from $(-9, 1.5)$ and take 100 steps. SGD and SGD with momentum ($\mu = 0.9$) use $\eta = 0.035$, just below the stability limit $2/50$. Adam uses $\eta = 0.3$.

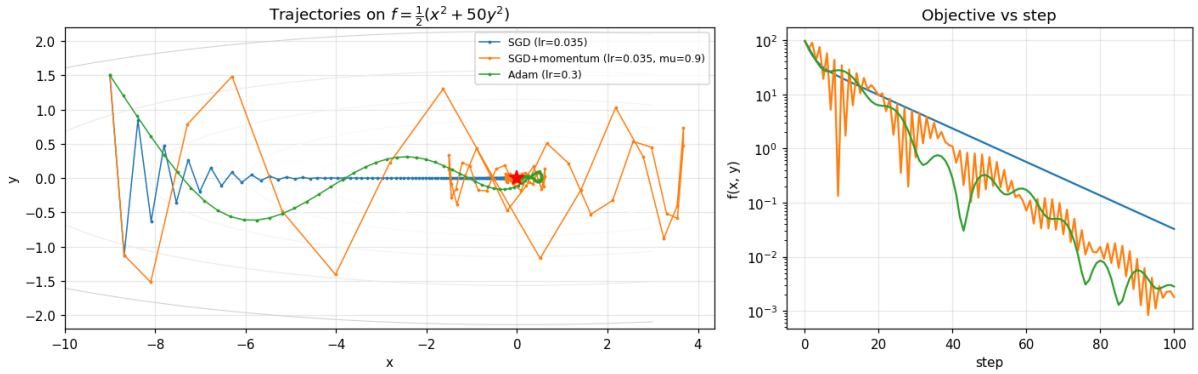


Figure 4: Left: trajectories on the contours of f. Right: f vs step (log scale).

Optimiser	f after 25 steps	50 steps	100 steps
SGD	6.82	1.15	3.26×10^{-2}
SGD + momentum	6.67	0.695	$\mathbf{1.80 \times 10^{-3}}$
Adam	4.70	0.296	2.82×10^{-3}

Interpretation. The Hessian is $\mathrm{diag}(1, 50)$, so the problem has condition number 50.

- **SGD:** in y the update factor is $1 - 50\eta = -0.75$, so the iterate *zig-zags* across the narrow valley. In x the factor is 0.965, so SGD then *crawls* along the flat valley floor. The learning rate cannot be increased, because $\eta > 2/50$ makes y diverge. This is how ill-conditioning slows gradient descent.
- **SGD with momentum** builds up velocity along the consistent x direction (effective step $\approx \eta/(1-\mu)$), while the alternating y gradients partly cancel. At this learning rate it is under-damped and overshoots (to $x \approx 3.7$), but it still reaches an objective 18× lower than SGD.
- **Adam** normalises each coordinate by the RMS of its gradient, so both coordinates move at a similar speed regardless of curvature. The path is a smooth curve towards the optimum, with no zig-zag. With a constant learning rate it oscillates slightly near the minimum, so f plateaus around 10^{-3}.

(c) Learning-rate sensitivity of a small CNN

The small CNN has three stages, each [3×3 conv, ReLU, 2×2 max-pool] with 32/64/128 channels, followed by an FC head (2048–256–10), with **no batch norm** (620k parameters). Each run uses a constant learning rate, batch 128, 15 epochs and 45k training images.

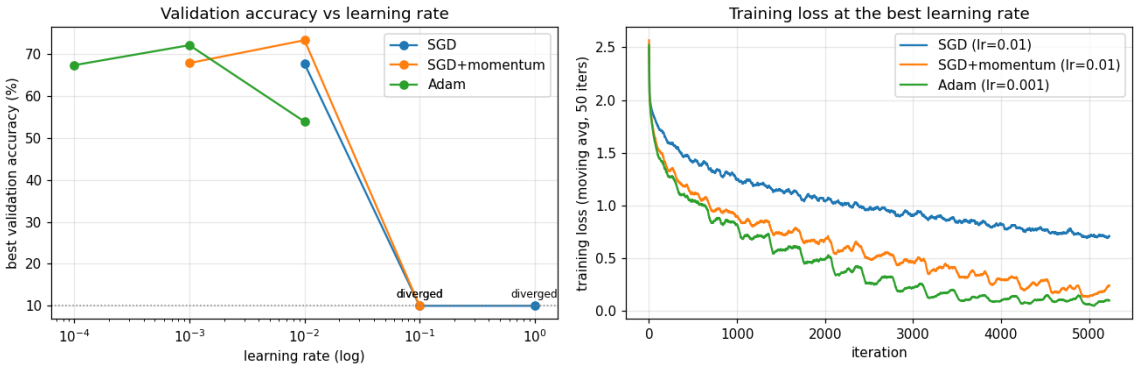


Figure 5: Left: best validation accuracy vs learning rate. Right: training loss at each optimiser's best learning rate.

Optimiser (learning rates)	low	mid	high
SGD (10^{-2}, 10^{-1}, 1)	**67.7**	10.0 (diverged)	10.0 (diverged)
SGD + momentum (10^{-3}, 10^{-2}, 10^{-1})	67.8	**73.3**	10.0 (diverged)
Adam (10^{-4}, 10^{-3}, 10^{-2})	67.3	**72.1**	53.9

Table 3: Best validation accuracy (%).

Interpretation.

- **Plain SGD** is the most fragile. Without batch norm, lr = 0.1 already blows up to NaN in the first epoch, and the only stable rate is too small for fast progress.
- **Momentum** multiplies the effective step by $1/(1-\mu) = 10$, so its usable range shifts down by about a decade. SGD-m at 10^{-3} behaves almost exactly like SGD at 10^{-2} (67.8 vs 67.7%). It is the best overall at 10^{-2} (73.3%) but diverges at 10^{-1}.
- **Adam** is the least sensitive. Nothing diverges over a 100× range, and even lr = 10^{-2} reaches 54%, because its per-parameter normalisation bounds the step to about lr.
- At the best rates, Adam reduces the training loss fastest (98% train accuracy by epoch 15), but SGD-m generalises slightly better. Adam's validation loss starts rising after about epoch 5.

In every case, the learning rate is the most important hyper-parameter and must be tuned on a log scale. Adaptive methods widen the safe range but do not remove the need to tune.

3 Task 3: Regularisation and Residual Connections

(a) Regularisation on 5k training images

The CNN here has two [conv–BN–ReLU] layers per stage (32/64/128 channels) and an FC head (2048–256–10), 815k parameters. It is trained on only 5k images for 60 epochs with SGD-momentum (lr 0.05, cosine decay). Settings compared: weight decay 5×10^{-4}; dropout 0.5 in the head; random flip with a 4-pixel padded crop; cutout (one 16×16 square); mixup ($\alpha = 1$); and all of these combined. Cutout and mixup are my own implementations:

```python
def cutout(x, size=16):          # zero a size x size square at a random centre (
    vectorised)
    B, C, H, W = x.shape
    cy = torch.randint(0, H, (B, 1, 1), device=x.device); cx = torch.randint(0, W, (B,
     1, 1), device=x.device)
    ys = torch.arange(H, device=x.device).view(1, H, 1); xs = torch.arange(W, device=x
    .device).view(1, 1, W)
    inside = (ys >= cy - size//2) & (ys < cy + size//2) & (xs >= cx - size//2) & (xs <
     cx + size//2)
    return x * (~inside).unsqueeze(1).float()          # 0 = channel mean after
    normalisation

def mixup(x, y, alpha=1.0):     # loss = lam*CE(out, y) + (1-lam)*CE(out, y[p])
    lam = float(np.random.beta(alpha, alpha))
    p = torch.randperm(x.size(0), device=x.device)
    return lam * x + (1 - lam) * x[p], y, y[p], lam
```

Setting	Train acc. (%)	Val. acc. (%)	Gap	Best val.
Baseline	99.98	61.06	38.92	61.24
Weight decay	100.00	63.08	36.92	64.10
Dropout	79.74	61.24	18.50	61.42
Flip + crop	79.78	**67.24**	12.54	**67.44**
Cutout	99.04	65.50	33.54	65.56
Mixup	78.72	59.82	18.90	60.40
All combined	46.52	45.72	**0.80**	46.70

Table 4: Final-epoch accuracies after 60 epochs on 5k images.

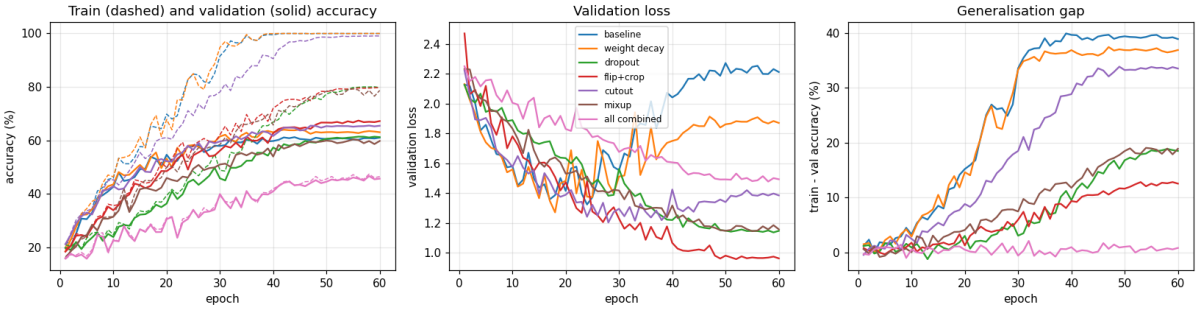


Figure 6: Left: train (dashed) and validation (solid) accuracy. Middle: validation loss. Right: generalisation gap.

Interpretation. The baseline memorises the 5k images (100% train accuracy by about epoch 40), while validation accuracy stalls at 61% and validation loss rises after epoch 20: classic overfitting with a 39-point gap.

- **Weight decay** gives only a small gain. With batch norm, conv weights are scale-invariant, so L2 mainly raises the effective learning rate.
- **Dropout** halves the gap but does not improve validation accuracy, because it regularises only the head, not the conv features.
- **Flip + crop** is clearly best (67.2%, gap 12.5, lowest validation loss). It encodes true invariances of natural images and so effectively enlarges the dataset.

- **Cutout** also helps (+4.4) by forcing the network to use the whole object, but the model still nearly memorises the data.
- **Mixup** narrows the gap but converges slowly on its soft targets (training loss still 1.4 at epoch 60) and needs longer training to pay off.
- **All combined** has almost no gap, but it *under-fits*: it cannot even fit the training set (46.5%) within 60 epochs.

A small gap is therefore not the goal in itself. The amount of regularisation must match model capacity and training time, and here label-preserving geometric augmentation gives the best trade-off.

(b) Plain vs residual networks of depth $6n + 2$

The CIFAR-style networks have a 3×3 stem, then three stages of n basic blocks (16/32/64 channels, stride 2 at the start of stages 2 and 3), global average pooling and an FC layer, with batch norm and Kaiming initialisation. The *only* difference between the two families is the shortcut. I use parameter-free "option A" shortcuts (subsample and zero-pad the channels), so plain and residual networks have identical parameter counts: 75k, 270k and 853k for depths 8, 20 and 56. Training: SGD-momentum, lr 0.1 with cosine decay, weight decay 10^{-4}, flip+crop, 30 epochs on 45k images.

```
class Block(nn.Module):
    def forward(self, x):
        out = self.b2(self.c2(F.relu(self.b1(self.c1(x)))))
        if self.shortcut:                                    # the ONLY difference
            s = x[:, :, ::self.stride, ::self.stride] if self.stride > 1 else x
            if self.pad > 0:                                 # option A: zero-pad new
    channels
                s = F.pad(s, (0, 0, 0, 0, self.pad // 2, self.pad - self.pad // 2))
            out = out + s
        return F.relu(out)
```

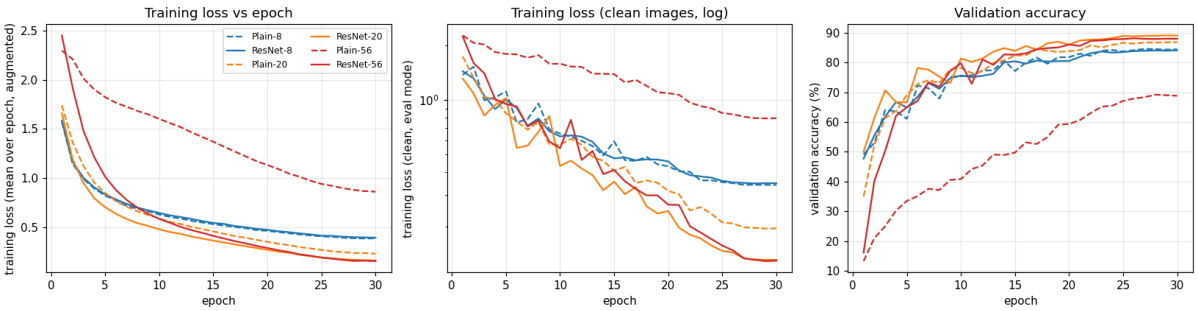


Figure 7: Plain (dashed) vs residual (solid) networks of depth 8, 20 and 56. Left: training loss vs epoch. Middle: training loss on clean images (log scale). Right: validation accuracy.

Network	Final training loss	Train acc. (%)	Val. acc. (%)
Plain-8	0.388	88.06	84.42
ResNet-8	0.396	87.76	84.10
Plain-20	0.232	93.16	86.82
ResNet-20	0.164	95.28	**89.02**
Plain-56	**0.862**	72.10	68.82
ResNet-56	**0.157**	95.60	87.96

Interpretation.

- **Depth 8:** the two families are identical, because a shallow network is easy to optimise.
- **Depth 20:** the shortcut already helps (loss 0.164 vs 0.232, validation +2.2 points).
- **Depth 56:** the plain network shows the **degradation problem**. Its *training* loss (0.86) is far higher than that of the shallower plain networks (0.23 and 0.39), so this is an optimisation failure, not overfitting. A 56-layer plain net could in principle copy the 20-layer solution with identity layers on top, but SGD cannot find that solution. Even with batch norm and Kaiming initialisation, the composition of 54 non-linear layers gives a poorly conditioned loss surface.
- **ResNet-56** trains easily and reaches the lowest loss of all (0.157). Each block computes $x+F(x)$, so representing the identity only requires $F \to 0$, and the gradient $\partial x_{l+1}/\partial x_l = I + \partial F/\partial x_l$ gives every layer a direct path to the loss.

ResNet-56 does not beat ResNet-20 on validation accuracy within 30 epochs. Deeper networks typically need longer schedules to realise their advantage.

4 Task 4: Best CNN with ≤500k Parameters and ≤20 Epochs

Design. A residual network (`CifarNet(n=2, widths=(24,48,96))`): a 3×3 stem, then 3 stages of 2 basic blocks with 24/48/96 channels, option-A shortcuts, batch norm, Kaiming initialisation and global average pooling. It has **386,626 parameters**. Training: SGD with momentum 0.9, a one-cycle learning-rate schedule (peak 0.1), weight decay 5×10^{-4}, label smoothing 0.1, flip + crop + cutout augmentation, batch 128, 20 epochs.

Ablation. One design choice is changed at a time. Each run trains on the 45k split for 20 epochs and is compared on the validation split.

Configuration	Train acc. (%)	Val. acc. (%)	Δ vs full
Full model	91.84	**88.50**	—
No shortcut (plain)	91.36	88.36	−0.14
No augmentation	100.00	85.96	**−2.54**
No weight decay	91.12	86.40	−2.10
Adam (peak lr 3×10^{-3}) instead of SGD	90.68	87.82	−0.68

Table 5: Ablation study (validation split).

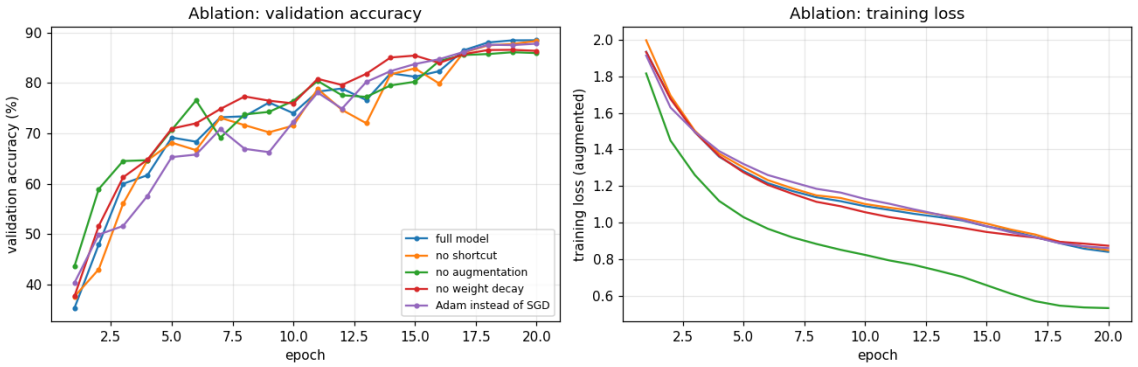


Figure 8: Ablation: validation accuracy (left) and training loss on augmented data (right).

Final model and test result. The full configuration had the best validation accuracy. It was retrained on the 45k split for 20 epochs, and the test set was evaluated **once**.

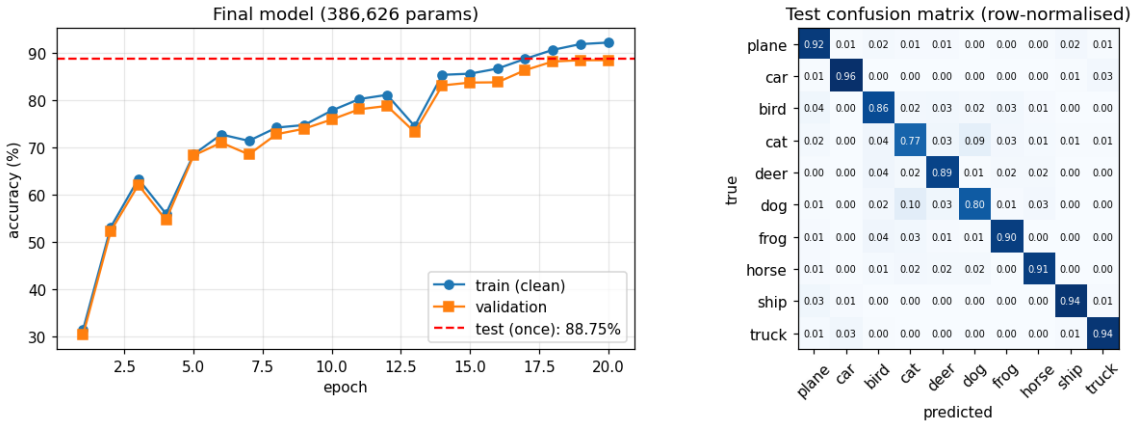


Figure 9: Final model: accuracy vs epoch with the single test evaluation (left), and row-normalised test confusion matrix (right).

Final model: 386,626 parameters, 20 epochs
Validation accuracy 88.38% | **Test accuracy 88.75%**

Interpretation.

- **Augmentation** has the largest effect (−2.5 points without it). Without it the model reaches 100% train accuracy and overfits, which is consistent with Task 3(a). Its training loss is also much lower in the right-hand plot, because the task becomes easier.
- **Weight decay** is the second most important choice (−2.1). With batch norm it keeps weight norms small and therefore keeps the effective learning rate high late in training.
- **Adam** is slightly worse (−0.7). SGD-momentum with weight decay generalises better here, and part of the difference is because Adam's L2 decay is coupled (AdamW would be fairer).
- **Removing the shortcut** makes no significant difference (−0.14) at only 14 layers, matching Task 3(b), where shortcuts mattered only at depth 56. The retrained full model reached 88.38% validation vs 88.50% in the ablation run with the same configuration. Differences of about ±0.2 points are therefore run-to-run noise from GPU nondeterminism.

Test accuracy (88.75%) is very close to validation accuracy, so tuning did not overfit the validation split. The confusion matrix shows that cars, ships and trucks are easiest (94–96%), while **cat (77%) and dog (80%)** are hardest, mainly confused with each other because they look alike at 32×32 resolution.

Summary

- The hand-written backpropagation and optimisers match autograd and `torch.optim` to machine precision.
- Sigmoid networks suffer vanishing gradients; ReLU with He initialisation preserves gradient scale.
- Momentum and Adam overcome ill-conditioning. Adam is the most robust to the learning rate, while SGD-momentum gave the best accuracy.
- Geometric augmentation is the most effective regulariser; too much regularisation causes under-fitting.
- Residual connections fix the degradation of deep plain networks.
- The final 387k-parameter ResNet reaches **88.75%** test accuracy within 20 epochs.